

第 9 讲: 损失函数

模型如何衡量预测误差并指导参数更新

@ Frano

● FranSpace

本讲摘要 / Why it matters

损失函数 (Loss Function) 是神经网络训练中的评价标准。前向传播给出预测结果, 损失函数负责衡量预测结果和目标之间的差距, 再由反向传播计算梯度, 最后由优化器更新参数。对普通监督学习来说, 损失函数通常衡量预测值和标签之间的误差; 对 PINN 来说, 损失函数还要衡量控制方程残差、边界条件误差、初始条件误差以及观测数据误差。因此, 损失函数设计是否合理, 会直接影响模型能否学到正确的物理规律。

9.1 为什么需要损失函数

神经网络的训练目标不是简单地“让网络输出一个数”, 而是希望输出结果尽量接近我们想要的目标。问题是, 计算机并不知道“接近”是什么意思, 也不知道当前预测结果是好还是坏。损失函数的作用, 就是把这种“好坏”转化成一个可以计算的数值。

假设真实值是 y , 模型预测值是 $\hat{y}$ 。如果二者完全相等, 那么预测误差为 0; 如果二者相差很大, 那么预测误差就大。损失函数就是把这种误差转化为训练目标:

$$L = L(\hat{y}, y).$$

在神经网络训练中, 损失函数越小, 通常说明模型预测越接近目标。训练模型的过程可以理解为不断调整参数 θ , 使损失函数逐渐减小:

$$\theta^* = \arg \min_{\theta} L(\theta).$$

分析 / Insight

损失函数相当于神经网络训练中的“评分标准”。如果评分标准设计合理，模型就有可能朝正确方向学习；如果评分标准设计错误，模型即使把 loss 降得很低，也可能得到错误结果。

9.2 损失函数在训练流程中的位置

在一个完整的训练循环中，损失函数处在前向传播和反向传播之间。前向传播先根据当前参数得到预测结果，损失函数把预测结果和目标进行比较，反向传播再根据损失函数计算梯度。

损失函数把“预测结果好不好”转换成一个优化目标



这个流程可以写成：

$$\mathbf{x} \xrightarrow{\text{前向传播}} \hat{\mathbf{y}} \xrightarrow{\text{损失函数}} L \xrightarrow{\text{反向传播}} \nabla_{\theta} L \xrightarrow{\text{优化器}} \theta_{\text{new}}.$$

因此，损失函数不仅是一个误差指标，也是连接“模型预测”和“参数更新”的桥梁。没有损失函数，反向传播就不知道应该计算哪个目标的梯度；优化器也不知道参数应该朝哪个方向更新。

9.3 从一个简单例子理解损失

先看一个最简单的一维回归问题。假设真实关系大致是

$$y = 2x + 1,$$

但模型当前预测为

$$\hat{y} = wx + b.$$

如果某个样本的真实值是 $y = 5$ ，模型预测值是 $\hat{y} = 3$ ，那么误差为

$$e = \hat{y} - y = 3 - 5 = -2.$$

这个误差本身有正负号。正号表示预测偏大，负号表示预测偏小。如果直接把所有误差相加，正负误差可能互相抵消，所以通常不会直接使用

$$\sum_i (\hat{y}_i - y_i)$$

作为损失函数。

更常见的做法是把误差变成非负数，例如平方或取绝对值。这样只要预测不准，损失就会增加。

Example / 例子

如果两个样本的误差分别是 +2 和 -2，直接相加得到 0，看起来似乎没有误差。但实际上两个样本都预测错了。因此，损失函数通常需要避免正负误差相互抵消。

9.4 均方误差

均方误差 (Mean Squared Error, MSE) 是回归问题中最常见的损失函数之一。它的形式为

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

如果令误差为

$$e_i = \hat{y}_i - y_i,$$

那么 MSE 就是所有误差平方的平均值：

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N e_i^2.$$

MSE 的特点是对大误差非常敏感。因为误差被平方后，大误差会被放大。例如误差从 1 增加到 2，平方误差从 1 增加到 4；误差从 2 增加到 4，平方误差从 4 增加到 16。这意味着 MSE 会强烈惩罚偏差较大的样本。

MSE 的优点是形式光滑、便于求导，因此非常适合基于梯度的优化方法。但它也有缺点：如果数据中有异常值，MSE 会对异常值特别敏感，因为大误差会被平方放大。

9.5 平均绝对误差

平均绝对误差 (Mean Absolute Error, MAE) 的形式为

$$L_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

MAE 直接取误差绝对值。它的优点是直观，单位和原始物理量相同。例如如果预测位移的单位是 mm，那么 MAE 的单位也是 mm。相比 MSE，MAE 对异常值没有那么敏感，因为它不会把大误差平方放大。

但 MAE 的缺点是，在误差为 0 的位置不可导。虽然实际深度学习框架可以通过次梯度等方式处理，但从优化角度看，它不如 MSE 那样平滑。

备注 / Note

MSE 和 MAE 都常用于回归问题。MSE 更强调大误差，优化更平滑；MAE 更直观，对异常值更稳健。实际任务中可以根据目标选择，也可以同时报告二者作为误差评价指标。

9.6 分类问题中的交叉熵损失

对于分类问题，模型输出通常不是一个连续值，而是类别概率。此时常用交叉熵损失 (Cross Entropy Loss)。以二分类为例，如果真实标签为 $y \in \{0, 1\}$ ，模型预测正类概率为 p ，则二分类交叉熵为

$$L_{\text{BCE}} = -[y \log p + (1 - y) \log(1 - p)].$$

如果是真实类别为 k 的多分类问题，模型输出第 j 类概率为 p_j ，则交叉熵可以写成

$$L_{\text{CE}} = -\log p_k.$$

交叉熵的含义是：如果模型给真实类别很高概率，则损失小；如果模型给真实类别很低概率，则损失大。它特别适合分类问题，因为它直接衡量概率分布之间的差异。

不过，在固体力学 PINN 问题中，主要变量通常是位移、应力、能量或残差等连续物理量，因此 MSE 型损失会更常见。

9.7 损失函数和误差指标的区别

初学时容易把“损失函数”和“误差指标”混在一起。它们确实都在衡量模型表现，但用途不同。

损失函数主要用于训练。它必须适合反向传播和梯度优化，因此通常需要可导或几乎处处可导。例如训练时使用 MSE，是因为它光滑、稳定、容易求梯度。

误差指标主要用于评价。它不一定直接用于训练。例如在固体力学 PINN 中，我们可以训练时使用总损失 L ，但评价时分别报告位移 MAE、应力相对误差、最大误差、PDE 残差云图等。

也就是说：

训练目标 $\neq$ 最终评价指标。

分析 / Insight

对 PINN 来说，这一点尤其重要。总 loss 下降不代表位移、应力、边界反力都正确。因此，训练时需要关注损失，评估时还要单独检查物理量误差。

9.8 普通监督学习中的损失函数

在普通监督学习中，训练数据通常是一组输入和标签：

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N.$$

模型预测为

$$\hat{\mathbf{y}}_i = f_{\theta}(\mathbf{x}_i).$$

此时最常见的数据损失可以写成

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|^2.$$

这个损失的目标很直接：让模型预测尽量接近标签。如果数据足够多、标签足够准确，而且模型表达能力足够强，那么监督学习可以取得很好的效果。

但对于科学计算问题，尤其是 PINN，很多时候我们没有大量标签数据。例如，我们可能没有每个点的真实位移和应力，但我们知道它们应该满足控制方程和边界条件。这就需要把物理约束也写进损失函数。

9.9 PINN 中的损失函数

物理信息神经网络 (Physics-Informed Neural Network, PINN) 的核心不是只拟合数据，而是让网络预测的函数同时满足物理方程、边界条件和必要的数据约束。

以固体力学问题为例，如果网络预测位移场

$$\mathbf{u}_{\theta}(\mathbf{X}) = \begin{bmatrix} u_{\theta}(\mathbf{X}) \\ v_{\theta}(\mathbf{X}) \end{bmatrix},$$

那么我们可以通过自动微分计算位移梯度、变形梯度、应力以及平衡方程残差。总损失通常由多个部分组成：

$$L = \lambda_r L_r + \lambda_D L_D + \lambda_N L_N + \lambda_{\text{data}} L_{\text{data}}.$$

其中：

- L_r 表示 PDE 残差损失；
- L_D 表示 Dirichlet 边界损失；

- L_N 表示 Neumann 边界损失;
- L_{data} 表示观测数据或参考解损失;
- $\lambda_r, \lambda_D, \lambda_N, \lambda_{data}$ 是各项权重。

PINN中的损失函数通常由多个物理约束项共同构成

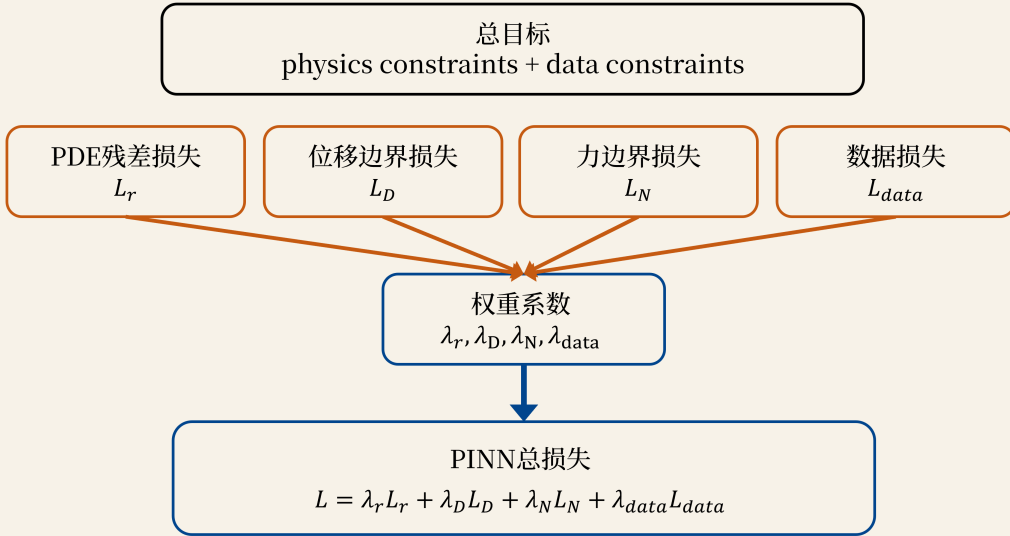


图 1: PINN 总损失的组成示意图

9.10 PDE 残差损失

偏微分方程 (Partial Differential Equation, PDE) 残差损失用于衡量网络预测函数是否满足控制方程。以静力平衡问题为例，参考构形下的平衡方程可以写成

$$\text{Div } \mathbf{P} + \mathbf{B} = \mathbf{0}.$$

如果网络预测位移后计算得到应力 $\mathbf{P}_\theta$ ，则残差可以写成

$$\mathbf{r}_\theta(\mathbf{X}) = \text{Div } \mathbf{P}_\theta(\mathbf{X}) + \mathbf{B}(\mathbf{X}).$$

PDE 残差损失通常写成

$$L_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \|\mathbf{r}_\theta(\mathbf{X}_i)\|^2.$$

它的物理含义是：在内部残差点上，网络预测的位移场经过本构关系和自动微分后，是否满足平衡方程。若 L_r 很小，说明内部平衡方程在采样点上满足较好。

9.11 Dirichlet 边界损失

Dirichlet 边界条件通常用于约束位移，例如

$$\mathbf{u} = \bar{\mathbf{u}} \quad \text{on } \Gamma_D.$$

对应的损失可以写成

$$L_D = \frac{1}{N_D} \sum_{i=1}^{N_D} \|\mathbf{u}_\theta(\mathbf{X}_i) - \bar{\mathbf{u}}(\mathbf{X}_i)\|^2.$$

它的作用是让网络预测的位移在指定边界上接近给定值。对固体力学问题而言，位移边界条件非常重要。如果位移边界没有满足，即使 PDE 残差很小，最终解也可能完全不对。

Dirichlet 条件也可以通过硬约束方式施加。例如在 $X = 0$ 上要求 $u = 0$ ，可以构造

$$u_\theta(X, Y) = X \hat{u}_\theta(X, Y).$$

这种做法使边界条件天然满足，不需要或少需要额外的位移边界损失。

9.12 Neumann 边界损失

Neumann 边界条件用于约束力或牵引边界。对于参考构形下的有限变形固体力学问题，常见形式是

$$\mathbf{PN} = \bar{\mathbf{T}} \quad \text{on } \Gamma_N.$$

对应的损失可以写成

$$L_N = \frac{1}{N_N} \sum_{i=1}^{N_N} \|\mathbf{P}_\theta(\mathbf{X}_i)\mathbf{N}_i - \bar{\mathbf{T}}(\mathbf{X}_i)\|^2.$$

Neumann 边界损失比 Dirichlet 边界损失更容易出错，因为它涉及应力计算、法向量方向、应力类型以及参考构形和当前构形的区别。在固体力学 PINN 中，如果力边界条件写错，可能出现总 loss 下降但位移和应力结果完全不对的情况。

备注 / Note

对固体力学 PINN 问题来说，Neumann 边界损失需要特别小心。必须确认使用的是第一类 Piola-Kirchhoff 应力还是 Cauchy 应力，法向量是在参考构形还是当前构形中定义，牵引方向和符号是否与有限元算例一致。

9.13 数据损失

如果有实验数据、有限元参考解或解析解，可以加入数据损失：

$$L_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \|\mathbf{u}_{\theta}(\mathbf{X}_i) - \mathbf{u}_{\text{ref}}(\mathbf{X}_i)\|^2.$$

数据损失的作用是用已知结果直接约束网络预测。它可以提高训练稳定性，也可以帮助网络在某些关键区域更接近参考解。

不过，数据损失也需要谨慎使用。如果参考数据本身存在插值误差，或者应力类型不一致，就可能把错误信息引入训练。对 PINN 来说，数据项应当和物理约束项保持一致。

9.14 为什么 PINN 的 loss 很难平衡

PINN 中的损失函数比普通监督学习复杂得多，因为它通常由多个不同物理意义的项组成。这些项的数量级可能差别很大。例如：

- PDE 残差可能是 10^2 量级；
- 位移边界损失可能是 10^{-4} 量级；
- 力边界损失可能是 10^1 量级；
- 数据损失可能又是另一种尺度。

如果直接相加，数值大的项会主导训练，数值小的项可能几乎不起作用。因此，PINN 通常需要引入 loss 权重：

$$L = \lambda_r L_r + \lambda_D L_D + \lambda_N L_N + \lambda_{\text{data}} L_{\text{data}}.$$

这些权重的作用不是装饰，而是控制不同物理约束在训练中的相对重要性。

分析 / Insight

PINN 中常见的“loss 一直下降但结果不对”，很多时候并不是网络完全不会学习，而是总损失中某些项被忽略了。例如 PDE 残差下降了，但 Neumann 边界没有真正满足，最终力边界问题仍然会错。

9.15 固定权重与自适应权重

最简单的做法是人为设定固定权重，例如

$$\lambda_r = 1, \quad \lambda_D = 100, \quad \lambda_N = 10.$$

这种方法实现简单，但问题是不同算例的损失尺度不一样，很难找到一组通用权重。

因此，PINN 中常常使用自适应权重方法，例如：

- 根据各项 loss 的下降速度调整权重；
- 根据梯度范数调整权重；
- 根据相对 loss 变化调整权重；
- 根据训练过程中不同约束的收敛速度动态调整权重。

后面学习 SoftAdapt、ReLoBRaLo、GradNorm 等方法时，都会围绕这个问题展开：如何让不同 loss 项都被充分训练，而不是只优化其中一部分。

9.16 损失函数设计中的常见错误

损失函数是 PINN 中最容易隐藏错误的地方。常见问题包括：

- 只看总 loss，不分别查看每个子 loss；
- PDE 残差公式写错；
- 边界点采样错误；
- Neumann 边界法向量方向错误；
- 应力类型混用；
- loss 权重设置不合理；
- 不同物理量没有做合理归一化；
- 自动微分图被 detach 或 no_grad 截断。

这些问题都会导致一个危险现象：训练曲线看起来正常下降，但物理结果不正确。因此，在调试 PINN 时，不应该只打印总 loss，而应该分别打印每一个子项。

9.17 如何监控损失

训练时建议至少记录以下内容：

- 总损失 L ；
- PDE 残差损失 L_r ；
- Dirichlet 边界损失 L_D ；
- Neumann 边界损失 L_N ；

- 数据损失 L_{data} ;
- 位移误差;
- 应力误差;
- 残差云图。

其中, 前几项用于观察训练过程, 后几项用于判断物理结果是否真正正确。特别是固体力学问题中, 应力误差往往比位移误差更敏感, 因此不能只看位移趋势。

9.18 一页总结

- 损失函数 (Loss Function) 是神经网络训练中的评价标准, 它把预测误差转化为可优化的数值目标。
- MSE 对大误差惩罚更强, 形式光滑, 适合回归训练; MAE 更直观, 对异常值不那么敏感。
- 分类问题常用交叉熵损失, 回归问题常用 MSE 或 MAE。
- 损失函数主要用于训练, 误差指标主要用于评价, 二者不一定完全相同。
- PINN 的总损失通常包括 PDE 残差损失、Dirichlet 边界损失、Neumann 边界损失和数据损失。
- PINN 中不同 loss 项尺度可能差别很大, 因此 loss 权重非常重要。
- 总 loss 下降不代表物理结果一定正确, 必须分别检查每一个子 loss 和关键物理量误差。
- 对固体力学 PINN, Neumann 边界损失、应力误差和角点误差尤其需要单独关注。